

ASD Eye-Tracking — TabPFN Full Pipeline (All Bugs Fixed)

What was fixed vs the original code

| #  | Bug                              | Original                                                                                                                                                   | Fix                                                                                        |
|----|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1  | Saccade grouping broken          | <code>((sac[C_CAT_RIGHT] != sac[C_CAT_RIGHT].shift()).cumsum())</code> — every row is 'Saccade' so shift is always equal → all saccades merge into 1 event | Use <code>Index Right</code> per trial: <code>((sac[C_IDX_RIGHT] != sac[C_IDX_RIGHT</code> |
| 2  | Blink grouping broken            | Same bug — all blinks collapse into 1 giant event                                                                                                          | Same fix: use <code>Index Right</code>                                                     |
| 3  | Global sort scrambles trials     | <code>df.sort_values(C_TIME)</code> globally — RecordingTime is NOT monotonic across trials (separator rows cause time jumps)                              | Sort <b>within each trial</b> only, after <code>groupby(C_TRIAL)</code>                    |
| 4  | Median imputation leakage        | <code>df[feat_cols].fillna(df[feat_cols].median())</code> computed on ALL 57 participants before split                                                     | Split first, compute median on train only, apply to both                                   |
| 5  | Global feature selection leakage | <code>global_feature_validation(df_train, feat_cols)</code> uses all 46 train participants including the left-out fold participant                         | All feature selection moved <b>inside</b> each LOOCV fold only                             |
| 6  | Pupil column exact-match bug     | <code>any(p in f for p in PUPIL_COLS)</code> — substring match, e.g. <code>pupil_mean</code> matches <code>pupil_mean_norm</code>                          | Changed to <code>f in PUPIL_COLS</code> exact set lookup                                   |
| 7  | Fixation duration off-by-one     | <code>times[-1] - times[0]</code> underestimates by one sample interval                                                                                    | <code>times[-1] - times[0] + median_sample_interval</code>                                 |
| 8  | NaN in inter-fixation distance   | <code>np.diff(x)</code> with possible NaN coordinates                                                                                                      | Mask NaNs before diff                                                                      |
| 9  | MIN_FIXATIONS drops not tracked  | Silently drops trials, no QC output                                                                                                                        | Count and report dropped trials per participant and per class                              |
| 10 | Index Right resets per trial     | Code assumed Index Right was global across all trials                                                                                                      | Confirmed: resets per trial. Grouping done within <code>groupby(Trial)</code>              |

Run cells in order, top to bottom.

Cell 0 — Install dependencies

```
import os
os.environ["TABPFN_TOKEN"] = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjojMDM2ODE5OGItMDYwMS00NTE1LWJhMWMtMmUxYmUxYzhmMzU5IiwiaXhwIjojODAwNDU5NjE1fQ.T3wBttf1n1N3M6gDG7FCg6NTixU16V5r;
print("Token set. Length:", len(os.environ["TABPFN_TOKEN"]))
print("Dots:", os.environ["TABPFN_TOKEN"].count('.'))
```

Token set. Length: 167  
Dots: 2

```
import os
os.environ["TABPFN_TOKEN"] = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjojMDM2ODE5OGItMDYwMS00NTE1LWJhMWMtMmUxYmUxYzhmMzU5IiwiaXhwIjojODAwNDU5NjE1fQ.T3wBttf1n1N3M6gDG7FCg6NTixU16V5r;

from tabpfn import TabPFNClassifier
import numpy as np
X = np.random.rand(20, 5)
y = np.array([0]*10 + [1]*10)
m = TabPFNClassifier(n_estimators=4, device='cpu')
m.fit(X, y)
print("SUCCESS - token works!")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.  
WARNING:huggingface\_hub.utils.\_http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.

tabpfn-v2.6-classifier-v2.6\_default.ckpt: 100% 43.0M/43.0M [00:01<00:00, 58.0MB/s]

config.json: 100% 35.0/35.0 [00:00<00:00, 1.02kB/s]

SUCCESS - token works!

```
import os
token = "your-full-token-here"
os.environ["TABPFN_TOKEN"] = token
print(f"Token length: {len(token)}") # should be 200+ characters
print(f"Dot count: {token.count('.')}") # must be exactly 2
```

```
Token length: 20
Dot count: 0
```

```
# Cell 0 – Install all required packages
# Run this first, then restart runtime if prompted

!pip install tabpfn scikit-learn statsmodels pandas numpy matplotlib shap joblib scipy tqdm -q

# Verify TabPFN import
try:
    from tabpfn import TabPFNClassifier
    print('TabPFN imported OK')
except ImportError as e:
    print(f'TabPFN import failed: {e}')
    print('Try: pip install tabpfn --upgrade')
```

```
TabPFN imported OK
```

## ✓ Cell 1 — Mount Google Drive and set paths

```
# Cell 1 – Mount Drive and configure folder paths

from google.colab import drive
drive.mount('/content/drive')

import os

# — PATHS — matches your Drive structure: My Drive > asd_eye_dataset ———
BASE = '/content/drive/MyDrive/asd_eye_dataset'
ASD_FOLDER = os.path.join(BASE, 'ASD_degraded')
TD_FOLDER = os.path.join(BASE, 'TD_degraded')
OUTPUT_DIR = os.path.join(BASE, 'results_tabpfn_fixed')
FEATURE_FILE = os.path.join(OUTPUT_DIR, 'trial_features_tabpfn.csv')
SPLIT_FILE = os.path.join(OUTPUT_DIR, 'split_indices.json')
HOLDOUT_FLAG = os.path.join(OUTPUT_DIR, '.holdout_used')
# —————

os.makedirs(OUTPUT_DIR, exist_ok=True)

# Verify folders exist
for folder, name in [(ASD_FOLDER, 'ASD'), (TD_FOLDER, 'TD')]:
    if os.path.exists(folder):
        files = [f for f in os.listdir(folder) if f.lower().endswith('.csv')]
        print(f'{name} folder OK – {len(files)} CSV files')
    else:
        print(f'ERROR: {name} folder not found at {folder}')
        print('Please update the path above.')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True).  
 ASD folder OK – 27 CSV files  
 TD folder OK – 30 CSV files

## Cell 2 — All constants and column names

```
# Cell 2 – Constants. Do not change unless your CSV column names differ.

# — CSV column names (verified against participant_11.csv) —————
C_TIME      = 'RecordingTime [ms]'
C_CAT_GROUP = 'Category Group'
C_CAT_RIGHT = 'Category Right'
C_IDX_RIGHT = 'Index Right'
C_GAZE_RX   = 'Point of Regard Right X [px]'
C_GAZE_RY   = 'Point of Regard Right Y [px]'
C_GAZE_LX   = 'Point of Regard Left X [px]'
C_GAZE_LY   = 'Point of Regard Left Y [px]'
C_PUPIL_R   = 'Pupil Diameter Right [mm]'
C_PUPIL_L   = 'Pupil Diameter Left [mm]'
C_TRACK_RATIO = 'Tracking Ratio [%]'
C_PARTICIPANT = 'Participant'
C_TRIAL      = 'Trial'
C_STIMULUS   = 'Stimulus'

# — Pipeline thresholds —————
BLINK_SHORT_MS = 100 # blinks shorter than this are micro-blinks (still processed)
BLINK_LONG_MS  = 500 # threshold for 'long blink' feature
MIN_TRACK_RATIO = 50.0 # drop rows below this tracking quality
MAX_FIX_DUR_MS  = 10000 # fixations longer than this are artefacts
MIN_FIXATIONS   = 3 # trials with fewer fixations are dropped

# — Model / CV config —————
RANDOM_STATE = 42
ALPHA        = 0.05
N_ENSEMBLE   = 32 # 32 for dev; 64 for camera-ready
N_PERM       = 100 # 100 for dev; 1000 for camera-ready
N_BOOT       = 2000
HOLDOUT_RATIO = 0.20
META_COLS    = ['participant_id', 'trial', 'stimulus_type', 'label']

# — Pupil columns for scaling – EXACT match (Bug 6 fix) —————
PUPIL_COLS_EXACT = {
    'pupil_r_mean', 'pupil_r_std',
    'pupil_l_mean', 'pupil_l_std',
    'pupil_asymmetry_mean', 'pupil_asymmetry_std',
    'pupil_slope'
}

print('Constants loaded OK')

Constants loaded OK
```

## Cell 3 — Preprocessing helpers

```
# Cell 3 - load_and_clean, interpolate_blinks, zscore_pupil
# KEY FIX: sorting is done PER TRIAL inside process_participant, not globally here.

import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

def load_and_clean(path):
    """
    Load CSV and clean:
    - Convert '-' to NaN
    - Numeric coercion for numeric columns
    - Replace 0.0 with NaN for gaze + pupil (0 is invalid in iViewX/SMI output)
    - Keep only 'Eye' Category Group rows
    - Drop Separator rows
    - Drop low-quality tracking rows
    NOTE: NO global sort_values(C_TIME) here.
    RecordingTime is NOT monotonic across trials (verified on participant_11).
    Sorting is done per-trial inside process_participant.
    """
    df = pd.read_csv(path, low_memory=False)

    # Replace '-' string with NaN everywhere except category columns
    cat_cols = [C_CAT_GROUP, C_CAT_RIGHT, 'Category Left']
    non_cat = [c for c in df.columns if c not in cat_cols]
    df[non_cat] = df[non_cat].replace('-', np.nan)

    # Numeric coercion
    for col in [C_TIME, C_IDX_RIGHT, C_GAZE_RX, C_GAZE_RY,
                C_GAZE_LX, C_GAZE_LY, C_PUPIL_R, C_PUPIL_L, C_TRACK_RATIO]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors='coerce')

    # 0.0 is invalid for gaze coordinates and pupil in this tracker
    for col in [C_PUPIL_R, C_PUPIL_L, C_GAZE_RX, C_GAZE_RY, C_GAZE_LX, C_GAZE_LY]:
        if col in df.columns:
            df[col] = df[col].replace(0.0, np.nan)

    # Keep only Eye rows (removes Information/Separator rows at the top level)
    if C_CAT_GROUP in df.columns:
        df = df[df[C_CAT_GROUP] == 'Eye'].copy()

    # Drop separator and NaN category rows
    if C_CAT_RIGHT in df.columns:
        df = df[~df[C_CAT_RIGHT].isin(['-', 'Separator'])].copy()
        df = df[df[C_CAT_RIGHT].notna()].copy()

    # Drop low tracking quality
    if C_TRACK_RATIO in df.columns:
        df = df[df[C_TRACK_RATIO] >= MIN_TRACK_RATIO].copy()
```

```

return df.reset_index(drop=True)

def interpolate_blinks(trial_df):
    """
    Within a single trial DataFrame:
    - For each blink episode >= BLINK_SHORT_MS, set gaze/pupil to NaN
    - Then linearly interpolate within the trial (limit_area='inside')
    Blink grouping uses Index Right (Bug 2 fix – same fix as saccades).
    """
    if C_CAT_RIGHT not in trial_df.columns or C_TIME not in trial_df.columns:
        return trial_df
    df = trial_df.copy()
    is_blink = df[C_CAT_RIGHT] == 'Blink'
    if not is_blink.any():
        return df

    # FIX Bug 2: use Index Right to group blink episodes
    blk = df[is_blink].copy()
    if C_IDX_RIGHT in blk.columns and blk[C_IDX_RIGHT].notna().sum() > 0:
        blink_groups = (blk[C_IDX_RIGHT] != blk[C_IDX_RIGHT].shift()).cumsum()
    else:
        # Fallback: group by time gaps > 100ms
        times = blk[C_TIME].values
        gaps = np.concatenate([[True], np.diff(times) > 100])
        blink_groups = pd.Series(gaps.cumsum(), index=blk.index)

    for _, grp in blk.groupby(blink_groups):
        times = grp[C_TIME].dropna().values
        if len(times) < 1:
            continue
        dur = (times[-1] - times[0]) if len(times) > 1 else 0.0
        if dur >= BLINK_SHORT_MS:
            for col in [C_GAZE_RX, C_GAZE_RY, C_GAZE_LX, C_GAZE_LY, C_PUPIL_R, C_PUPIL_L]:
                if col in df.columns:
                    df.loc[grp.index, col] = np.nan

    for col in [C_GAZE_RX, C_GAZE_RY, C_GAZE_LX, C_GAZE_LY, C_PUPIL_R, C_PUPIL_L]:
        if col in df.columns:
            df[col] = df[col].interpolate(method='linear', limit_area='inside')
    return df

def zscore_pupil(trial_df):
    """
    Z-score pupil within this trial/participant segment.
    Applied per-trial after the trial split to avoid cross-participant leakage.
    """
    df = trial_df.copy()
    for col in [C_PUPIL_R, C_PUPIL_L]:
        if col in df.columns:
            vals = df[col].dropna()
            if len(vals) > 1 and vals.std() > 0:
                df[col] = (df[col] - vals.mean()) / vals.std()
            else:
                df[col] = 0.0

```

```
return df
```

```
print('Preprocessing helpers defined OK')
```

```
Preprocessing helpers defined OK
```

## ✓ Cell 4 — Event builders (Bugs 1, 2, 7, 8 fixed)

```
# Cell 4 – build_fixation_events, build_saccade_events, build_blink_events
#
# BUG 1 FIX – saccade grouping:
# OLD: (sac[C_CAT_RIGHT] != sac[C_CAT_RIGHT].shift()).cumsum()
#      Every row is 'Saccade' so shift is always equal -> cumsum never increments
#      -> ALL saccades merge into one giant event
# NEW: (sac[C_IDX_RIGHT] != sac[C_IDX_RIGHT].shift()).cumsum()
#      Index Right changes between saccade events -> correct grouping
#
# BUG 2 FIX – blink grouping: same fix as Bug 1
#
# BUG 7 FIX – fixation duration off-by-one:
# OLD: times[-1] - times[0] (underestimates by one sample interval)
# NEW: times[-1] - times[0] + estimated_sample_interval
#
# BUG 8 FIX – NaN in inter-fixation distance: mask NaNs before np.diff
```

```
def safe_stat(arr, fn):
    """Apply fn to arr after removing NaNs. Returns 0.0 if empty."""
    if len(arr) == 0:
        return 0.0
    a = arr[~np.isnan(arr)]
    return float(fn(a)) if len(a) > 0 else 0.0
```

```
def build_fixation_events(trial_df):
    """
    Group fixation rows by Index Right (resets per trial – verified).
    Duration fix: add sample interval to avoid off-by-one underestimate.
    """
    fix = trial_df[trial_df[C_CAT_RIGHT] == 'Fixation'].copy()
    if len(fix) == 0:
        return pd.DataFrame()

    # Estimate sample interval from this trial
    all_times = trial_df[C_TIME].dropna().values
    if len(all_times) > 1:
        diffs = np.diff(all_times)
        diffs = diffs[diffs > 0]
        sample_interval = float(np.median(diffs)) if len(diffs) > 0 else 0.0
    else:
        sample_interval = 0.0

    # Group by Index Right (correct – resets per trial)
```

```

if C_IDX_RIGHT in fix.columns and fix[C_IDX_RIGHT].notna().sum() > 0:
    groups = (fix[C_IDX_RIGHT] != fix[C_IDX_RIGHT].shift()).cumsum()
else:
    # Fallback: group by time gaps
    times = fix[C_TIME].values
    gaps = np.concatenate([[True], np.diff(np.nan_to_num(times, nan=0)) > 200])
    groups = pd.Series(gaps.cumsum(), index=fix.index)

events = []
for seq, grp in fix.groupby(groups):
    times = grp[C_TIME].dropna().values
    if len(times) == 0:
        continue
    # BUG 7 FIX: add sample_interval to duration
    dur = float(times[-1] - times[0] + sample_interval) if len(times) > 1 else float(sample_interval)
    if dur > MAX_FIX_DUR_MS:
        continue
    x = grp[C_GAZE_RX].dropna().values if C_GAZE_RX in grp.columns else np.array([])
    y = grp[C_GAZE_RY].dropna().values if C_GAZE_RY in grp.columns else np.array([])
    pr = grp[C_PUPIL_R].dropna().values if C_PUPIL_R in grp.columns else np.array([])
    pl = grp[C_PUPIL_L].dropna().values if C_PUPIL_L in grp.columns else np.array([])
    events.append({
        'fix_seq': int(seq),
        'duration_ms': dur,
        'start_time': float(times[0]),
        'mean_x': float(np.mean(x)) if len(x) > 0 else np.nan,
        'mean_y': float(np.mean(y)) if len(y) > 0 else np.nan,
        'std_x': float(np.std(x)) if len(x) > 1 else 0.0,
        'std_y': float(np.std(y)) if len(y) > 1 else 0.0,
        'mean_pupil_r': float(np.mean(pr)) if len(pr) > 0 else np.nan,
        'mean_pupil_l': float(np.mean(pl)) if len(pl) > 0 else np.nan,
    })
return pd.DataFrame(events).sort_values('start_time').reset_index(drop=True)

def build_saccade_events(trial_df):
    """
    BUG 1 FIX: Group saccades by Index Right, not Category Right.
    Category Right is always 'Saccade' -> shift is always equal -> cumsum never increments.
    Index Right changes between saccade events -> correct grouping.
    Peak velocity excluded (unreliable at 28Hz after degradation).
    """
    sac = trial_df[trial_df[C_CAT_RIGHT] == 'Saccade'].copy()
    if len(sac) == 0:
        return pd.DataFrame()

    # FIX Bug 1: use Index Right
    if C_IDX_RIGHT in sac.columns and sac[C_IDX_RIGHT].notna().sum() > 0:
        groups = (sac[C_IDX_RIGHT] != sac[C_IDX_RIGHT].shift()).cumsum()
    else:
        # Fallback: time gaps > 50ms signal new saccade
        times = sac[C_TIME].values
        gaps = np.concatenate([[True], np.diff(np.nan_to_num(times, nan=0)) > 50])
        groups = pd.Series(gaps.cumsum(), index=sac.index)

    events = []

```

```

for _, grp in sac.groupby(groups):
    times = grp[C_TIME].dropna().values
    if len(times) == 0:
        continue
    x = grp[C_GAZE_RX].dropna().values if C_GAZE_RX in grp.columns else np.array([])
    y = grp[C_GAZE_RY].dropna().values if C_GAZE_RY in grp.columns else np.array([])
    amp = float(np.sqrt((x[-1]-x[0])**2 + (y[-1]-y[0])**2)) \
        if len(x) > 1 and len(y) > 1 else 0.0
    events.append({
        'duration_ms': float(times[-1]-times[0]) if len(times) > 1 else 0.0,
        'amplitude_px': amp,
    })
return pd.DataFrame(events)

def build_blink_events(trial_df):
    """
    BUG 2 FIX: Group blinks by Index Right, not Category Right.
    Same bug and same fix as saccades.
    """
    blk = trial_df[trial_df[C_CAT_RIGHT] == 'Blink'].copy()
    if len(blk) == 0:
        return pd.DataFrame()

    # FIX Bug 2: use Index Right
    if C_IDX_RIGHT in blk.columns and blk[C_IDX_RIGHT].notna().sum() > 0:
        groups = (blk[C_IDX_RIGHT] != blk[C_IDX_RIGHT].shift()).cumsum()
    else:
        times = blk[C_TIME].values
        gaps = np.concatenate([[True], np.diff(np.nan_to_num(times, nan=0)) > 100])
        groups = pd.Series(gaps.cumsum(), index=blk.index)

    events = []
    for _, grp in blk.groupby(groups):
        times = grp[C_TIME].dropna().values
        if len(times) == 0:
            continue
        dur = float(times[-1] - times[0]) if len(times) > 1 else 0.0
        events.append({'duration_ms': dur, 'is_long': int(dur >= BLINK_LONG_MS)})
    return pd.DataFrame(events)

print('Event builders defined OK')

```

Event builders defined OK

## ✓ Cell 5 — Feature extraction (31 features)

```

# Cell 5 - extract_trial_features
# BUG 8 FIX: NaN masking before np.diff in inter-fixation distance

from scipy.stats import linregress

```

```

def extract_trial_features(trial_df, fix_events, sac_events, blk_events):
    """
    Extract 31 features per trial.
    sac_peak_vel excluded (unreliable at 28Hz).
    gaze_x_std / gaze_y_std excluded (noise-inflated after degradation).
    """
    feats = {}

    times = trial_df[C_TIME].dropna().values if C_TIME in trial_df.columns else np.array([])
    rec_dur_min = max((times[-1] - times[0]) / 60_000.0, 0.001) \
        if len(times) > 1 else 1.0

    # — Fixation (10 features) —————
    if len(fix_events) > 0:
        durs = fix_events['duration_ms'].dropna().values
        feats['fix_count'] = len(fix_events)
        feats['fix_dur_mean_ms'] = safe_stat(durs, np.mean)
        feats['fix_dur_std_ms'] = safe_stat(durs, np.std)
        feats['fix_dur_max_ms'] = safe_stat(durs, np.max)
        feats['fix_dur_cv'] = feats['fix_dur_std_ms'] / (feats['fix_dur_mean_ms'] + 1e-8)
        feats['fix_rate_per_min'] = len(fix_events) / rec_dur_min
        disp = (fix_events['std_x'].fillna(0) + fix_events['std_y'].fillna(0)).values
        feats['fix_dispersion_mean'] = safe_stat(disp, np.mean)
        feats['fix_seq_norm'] = (fix_events['fix_seq'].mean() /
                                (fix_events['fix_seq'].max() + 1e-8))

        # BUG 8 FIX: mask NaNs in x, y before computing diff
        x_raw = fix_events['mean_x'].values
        y_raw = fix_events['mean_y'].values
        valid = ~np.isnan(x_raw) & ~np.isnan(y_raw)
        x_valid = x_raw[valid]
        y_valid = y_raw[valid]
        if len(x_valid) >= 2:
            ifd = np.sqrt(np.diff(x_valid)**2 + np.diff(y_valid)**2)
            feats['ifd_mean'] = float(np.mean(ifd))
            feats['ifd_std'] = float(np.std(ifd))
        else:
            feats['ifd_mean'] = 0.0
            feats['ifd_std'] = 0.0
    else:
        for k in ['fix_count', 'fix_dur_mean_ms', 'fix_dur_std_ms', 'fix_dur_max_ms',
                  'fix_dur_cv', 'fix_rate_per_min', 'fix_dispersion_mean', 'fix_seq_norm',
                  'ifd_mean', 'ifd_std']:
            feats[k] = 0.0

    # — Saccade (5 features) —————
    if len(sac_events) > 0:
        amps = sac_events['amplitude_px'].dropna().values
        feats['sac_count'] = len(sac_events)
        feats['sac_amplitude_mean'] = safe_stat(amps, np.mean)
        feats['sac_amplitude_std'] = safe_stat(amps, np.std)
        feats['sac_rate_per_min'] = len(sac_events) / rec_dur_min
        feats['sac_fix_ratio'] = len(sac_events) / (len(fix_events) + len(sac_events) + 1e-8)
    else:
        for k in ['sac_count', 'sac_amplitude_mean', 'sac_amplitude_std',
                  'sac_rate_per_min', 'sac_fix_ratio']:

```

```

    feats[k] = 0.0

# — Blink (4 features) —————
if len(blk_events) > 0:
    durs = blk_events['duration_ms'].dropna().values
    feats['blink_count'] = len(blk_events)
    feats['blink_dur_mean_ms'] = safe_stat(durs, np.mean)
    feats['blink_rate_per_min'] = len(blk_events) / rec_dur_min
    feats['blink_long_ratio'] = blk_events['is_long'].sum() / (len(blk_events) + 1e-8)
else:
    for k in ['blink_count', 'blink_dur_mean_ms', 'blink_rate_per_min', 'blink_long_ratio']:
        feats[k] = 0.0

```

```

# — Pupil (7 features) —————
for col, suf in [(C_PUPIL_R, 'r'), (C_PUPIL_L, 'l')]:
    if col in trial_df.columns:
        vals = trial_df[col].dropna().values
        feats[f'pupil_{suf}_mean'] = safe_stat(vals, np.mean)
        feats[f'pupil_{suf}_std'] = safe_stat(vals, np.std)
    else:
        feats[f'pupil_{suf}_mean'] = 0.0
        feats[f'pupil_{suf}_std'] = 0.0

if C_PUPIL_R in trial_df.columns and C_PUPIL_L in trial_df.columns:
    pr = trial_df[C_PUPIL_R].dropna()
    pl = trial_df[C_PUPIL_L].dropna()
    common = pr.index.intersection(pl.index)
    diff = (pr.loc[common] - pl.loc[common]).abs() if len(common) > 0 \
        else pd.Series([0])
    feats['pupil_asymmetry_mean'] = float(diff.mean())
    feats['pupil_asymmetry_std'] = float(diff.std()) if len(diff) > 1 else 0.0
else:
    feats['pupil_asymmetry_mean'] = 0.0
    feats['pupil_asymmetry_std'] = 0.0

```

```

if C_PUPIL_R in trial_df.columns:
    pvals = trial_df[C_PUPIL_R].dropna().values
    feats['pupil_slope'] = (float(linregress(range(len(pvals)), pvals).slope)
        if len(pvals) > 2 else 0.0)
else:
    feats['pupil_slope'] = 0.0

```

```

# — Gaze position (2 features) —————
if C_GAZE_RX in trial_df.columns and C_GAZE_RY in trial_df.columns:
    gx = trial_df[C_GAZE_RX].dropna().values
    gy = trial_df[C_GAZE_RY].dropna().values
    feats['gaze_x_mean'] = safe_stat(gx, np.mean)
    feats['gaze_y_mean'] = safe_stat(gy, np.mean)
else:
    feats['gaze_x_mean'] = 0.0
    feats['gaze_y_mean'] = 0.0

```

```

return feats

```

```

print('Feature extraction defined OK')

```

Feature extraction defined OK

## ✓ Cell 6 — Process one participant (Bug 3 fix: per-trial sort)

```
# Cell 6 – process_participant
#
# BUG 3 FIX – RecordingTime is NOT globally monotonic.
# Verified: drops occur at trial boundaries due to overlapping/restarted timestamps.
# FIX: sort within each trial group only, never globally.
#
# BUG 9 FIX – MIN_FIXATIONS drops are now tracked per participant and class.

def process_participant(path, label):
    """
    Returns (records, qc_info) where:
    - records: list of feature dicts (one per valid trial)
    - qc_info: dict with QC stats for this participant
    """
    qc = {'path': path, 'label': label, 'pid': None,
          'total_trials': 0, 'valid_trials': 0, 'dropped_trials': 0,
          'drop_reason': ''}

    # Extract participant ID from first few rows
    try:
        df_head = pd.read_csv(path, nrows=10, low_memory=False)
        pid = str(df_head[C_PARTICIPANT].dropna().iloc[0]) \
              if C_PARTICIPANT in df_head.columns else \
              os.path.splitext(os.path.basename(path))[0]
    except Exception:
        pid = os.path.splitext(os.path.basename(path))[0]
    qc['pid'] = pid

    try:
        df = load_and_clean(path)
    except Exception as e:
        print(f' ERROR loading {path}: {e}')
        qc['drop_reason'] = f'load_error: {e}'
        return [], qc

    if len(df) < 50:
        print(f' SKIP {pid} – too few rows after cleaning ({len(df)})')
        qc['drop_reason'] = 'too_few_rows'
        return [], qc

    if C_TRIAL not in df.columns:
        print(f' SKIP {pid} – no Trial column')
        qc['drop_reason'] = 'no_trial_column'
        return [], qc

    stim_map = df.groupby(C_TRIAL)[C_STIMULUS].first().to_dict() \
               if C_STIMULUS in df.columns else {}

    records = []
```

```

trial_names = df[C_TRIAL].unique()
qc['total_trials'] = len(trial_names)

for trial_name in trial_names:
    # BUG 3 FIX: sort within this trial only
    trial_df = df[df[C_TRIAL] == trial_name].copy()
    trial_df = trial_df.sort_values(C_TIME).reset_index(drop=True)

    # Blink interpolation and pupil z-scoring per trial
    trial_df = interpolate_blinks(trial_df)
    trial_df = zscore_pupil(trial_df)

    stim      = stim_map.get(trial_name, '')
    stim_type = 1 if str(stim).lower().endswith('.avi') else 0

    fix_ev = build_fixation_events(trial_df)
    sac_ev = build_saccade_events(trial_df)
    blk_ev = build_blink_events(trial_df)

    # BUG 9 FIX: track dropped trials
    if len(fix_ev) < MIN_FIXATIONS:
        qc['dropped_trials'] += 1
        continue

    feats = extract_trial_features(trial_df, fix_ev, sac_ev, blk_ev)
    feats.update({'participant_id': pid, 'trial': trial_name,
                  'stimulus_type': stim_type, 'label': label})
    records.append(feats)

qc['valid_trials'] = len(records)
return records, qc

```

```
print('process_participant defined OK')
```

```
process_participant defined OK
```

## ✓ Cell 7 — Run feature extraction for all participants

```

# Cell 7 – Extract features from all ASD and TD participants
# Saves trial_features_tabpfn.csv and a QC report.
# This is the most time-consuming cell (~5-15 min depending on dataset size).

import os
import sys

all_records = []
all_qc      = []

for folder, label, name in [(ASD_FOLDER, 1, 'ASD'), (TD_FOLDER, 0, 'TD')]:
    if not os.path.exists(folder):
        print(f'ERROR: {folder} not found')
        continue
    files = sorted(f for f in os.listdir(folder) if f.lower().endswith('.csv'))

```

```

print(f'\nProcessing {name} ({len(files)} participants)...')
for fname in files:
    fpath = os.path.join(folder, fname)
    print(f' {fname}', end=' ... ', flush=True)
    recs, qc = process_participant(fpath, label)
    all_records.extend(recs)
    all_qc.append(qc)
    print(f'OK ({qc["valid_trials"]} valid / {qc["total_trials"]} trials, '
          f'{qc["dropped_trials"]} dropped)')

if not all_records:
    print('No records extracted. Check your folder paths.')
else:
    df_out = pd.DataFrame(all_records)
    feat_cols = [c for c in df_out.columns if c not in META_COLS]
    df_out = df_out[META_COLS[:3] + feat_cols + ['label']]
    df_out.to_csv(FEATURE_FILE, index=False)
    print(f'\n{"="*55}')
    print(f' Saved: {FEATURE_FILE}')
    print(f' Participants: {df_out["participant_id"].nunique()}')
    print(f' Trials:      {len(df_out)} '
          f' ASD={int((df_out["label"]==1).sum())}, '
          f' TD={int((df_out["label"]==0).sum())}')
    print(f' Features:      {len(feat_cols)}')
    print(f' Missing:      {df_out[feat_cols].isnull().sum().sum()}')
    print(f'\n{"="*55}')

# BUG 9 FIX: QC report – show dropped trials by class
qc_df = pd.DataFrame(all_qc)
qc_df.to_csv(os.path.join(OUTPUT_DIR, 'qc_trial_drops.csv'), index=False)
print('\nQC – dropped trials by class:')
print(qc_df.groupby('label')[['total_trials', 'valid_trials', 'dropped_trials']].sum().to_string())
print(f'\nQC saved to: {os.path.join(OUTPUT_DIR, "qc_trial_drops.csv")}')

```

```

Processing ASD (27 participants)...
participant_1.csv ... OK (16 valid / 16 trials, 0 dropped)
participant_10.csv ... OK (18 valid / 18 trials, 0 dropped)
participant_11.csv ... OK (13 valid / 13 trials, 0 dropped)
participant_13.csv ... OK (18 valid / 18 trials, 0 dropped)
participant_14.csv ... OK (18 valid / 19 trials, 1 dropped)
participant_15.csv ... OK (21 valid / 21 trials, 0 dropped)
participant_17.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_18.csv ... OK (27 valid / 27 trials, 0 dropped)
participant_19.csv ... OK (26 valid / 26 trials, 0 dropped)
participant_2.csv ... OK (12 valid / 12 trials, 0 dropped)
participant_20.csv ... OK (28 valid / 29 trials, 1 dropped)
participant_21.csv ... OK (3 valid / 5 trials, 2 dropped)
participant_22.csv ... OK (29 valid / 29 trials, 0 dropped)
participant_23.csv ... OK (21 valid / 21 trials, 0 dropped)
participant_24.csv ... OK (33 valid / 33 trials, 0 dropped)
participant_25.csv ... OK (12 valid / 12 trials, 0 dropped)
participant_26.csv ... OK (12 valid / 12 trials, 0 dropped)
participant_27.csv ... OK (6 valid / 6 trials, 0 dropped)
participant_28.csv ... OK (11 valid / 11 trials, 0 dropped)
participant_29.csv ... OK (8 valid / 8 trials, 0 dropped)
participant_3.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_4.csv ... OK (18 valid / 19 trials, 1 dropped)

```

```

participant_5.csv ... OK (18 valid / 18 trials, 0 dropped)
participant_6.csv ... OK (17 valid / 17 trials, 0 dropped)
participant_7.csv ... OK (6 valid / 6 trials, 0 dropped)
participant_8.csv ... OK (11 valid / 11 trials, 0 dropped)
participant_9.csv ... OK (7 valid / 8 trials, 1 dropped)

```

Processing TD (30 participants)...

```

participant_30.csv ... OK (30 valid / 30 trials, 0 dropped)
participant_31.csv ... OK (31 valid / 31 trials, 0 dropped)
participant_32.csv ... OK (18 valid / 18 trials, 0 dropped)
participant_33.csv ... OK (20 valid / 20 trials, 0 dropped)
participant_34.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_35.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_36.csv ... OK (33 valid / 33 trials, 0 dropped)
participant_37.csv ... OK (20 valid / 20 trials, 0 dropped)
participant_38.csv ... OK (16 valid / 17 trials, 1 dropped)
participant_39.csv ... OK (20 valid / 20 trials, 0 dropped)
participant_40.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_41.csv ... OK (14 valid / 14 trials, 0 dropped)
participant_42.csv ... OK (13 valid / 15 trials, 2 dropped)
participant_43.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_44.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_45.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_46.csv ... OK (34 valid / 34 trials, 0 dropped)
participant_47.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_48.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_49.csv ... OK (30 valid / 30 trials, 0 dropped)
participant_50.csv ... OK (20 valid / 20 trials, 0 dropped)
participant_51.csv ... OK (19 valid / 19 trials, 0 dropped)
participant_52.csv ... OK (34 valid / 34 trials, 0 dropped)
participant_53.csv ... OK (34 valid / 34 trials, 0 dropped)
participant_54.csv ... OK (32 valid / 32 trials, 0 dropped)
participant_55.csv ... OK (32 valid / 32 trials, 0 dropped)

```

## ✓ Cell 8 — Load features, split participants, impute (all leakage fixed)

```

# Cell 8 – Load features and create participant split
#
# BUG 4 FIX – Median imputation leakage:
#   OLD: df[feat_cols].fillna(df[feat_cols].median()) <- computed on ALL 57
#   NEW: median computed on TRAIN only, then applied to both train and holdout
#
# This cell also handles BUG 9 (participant ID type mismatch: always cast to str)

import json
import datetime
from sklearn.preprocessing import StandardScaler

def load_features(path):
    """Load feature CSV, drop all-NaN columns and peak_vel column."""
    df = pd.read_csv(path, low_memory=False)
    df = df.dropna(axis=1, how='all')
    df = df[[c for c in df.columns if 'peak_vel' not in c]]
    df['participant_id'] = df['participant_id'].astype(str) # always str
    return df

```

```

def make_participant_split(df, holdout_ratio=HOLDOUT_RATIO, seed=RANDOM_STATE):
    """Stratified 80/20 participant split. Saves split JSON."""
    rng = np.random.default_rng(seed)
    parts = df[['participant_id', 'label']].drop_duplicates()
    asd_ids = parts[parts['label']==1]['participant_id'].tolist()
    td_ids = parts[parts['label']==0]['participant_id'].tolist()
    rng.shuffle(asd_ids); rng.shuffle(td_ids)
    n_hold_asd = max(1, round(len(asd_ids)*holdout_ratio))
    n_hold_td = max(1, round(len(td_ids)*holdout_ratio))
    holdout_ids = asd_ids[:n_hold_asd] + td_ids[:n_hold_td]
    train_ids = asd_ids[n_hold_asd:] + td_ids[n_hold_td:]
    split = {
        'train_ids': [str(x) for x in train_ids],
        'holdout_ids': [str(x) for x in holdout_ids],
        'seed': seed, 'holdout_ratio': holdout_ratio,
        'created': datetime.datetime.now().isoformat(),
        'n_train_asd': len(asd_ids)-n_hold_asd, 'n_train_td': len(td_ids)-n_hold_td,
        'n_hold_asd': n_hold_asd, 'n_hold_td': n_hold_td,
    }
    with open(SPLIT_FILE, 'w') as f:
        json.dump(split, f, indent=2)
    return train_ids, holdout_ids, split

# — Load —————
df = load_features(FEATURE_FILE)
feat_cols = [c for c in df.columns if c not in META_COLS]
print(f'Loaded: {df["participant_id"].nunique()} participants, '
      f'{len(df)} trials, {len(feat_cols)} features')

# — Split —————
if os.path.exists(SPLIT_FILE):
    with open(SPLIT_FILE) as f:
        split_info = json.load(f)
        train_ids = split_info['train_ids']
        holdout_ids = split_info['holdout_ids']
        print(f'Loaded existing split from {SPLIT_FILE}')
else:
    train_ids, holdout_ids, split_info = make_participant_split(df)
    print(f'Created new split -> {SPLIT_FILE}')

train_ids_s = [str(x) for x in train_ids]
holdout_ids_s = [str(x) for x in holdout_ids]

df_train_raw = df[df['participant_id'].isin(train_ids_s)].copy()
df_holdout_raw = df[df['participant_id'].isin(holdout_ids_s)].copy()

print(f' Train: {df_train_raw["participant_id"].nunique()} participants, '
      f'{len(df_train_raw)} trials '
      f'(ASD={split_info["n_train_asd"]}, TD={split_info["n_train_td"]})')
print(f' Holdout: {df_holdout_raw["participant_id"].nunique()} participants, '
      f'{len(df_holdout_raw)} trials '
      f'(ASD={split_info["n_hold_asd"]}, TD={split_info["n_hold_td"]})')

# — BUG 4 FIX: imputation medians from TRAIN only —————

```

```

train_medians = df_train_raw[feat_cols].median()
df_train      = df_train_raw.copy()
df_holdout    = df_holdout_raw.copy()
df_train[feat_cols] = df_train[feat_cols].fillna(train_medians)
df_holdout[feat_cols] = df_holdout[feat_cols].fillna(train_medians) # apply train medians

missing_train  = df_train[feat_cols].isnull().sum().sum()
missing_holdout = df_holdout[feat_cols].isnull().sum().sum()
print(f'\nAfter imputation (train medians only):')
print(f'  Train missing: {missing_train}')
print(f'  Holdout missing: {missing_holdout}')

Loaded: 57 participants, 1178 trials, 28 features
Loaded existing split from /content/drive/MyDrive/asd_eye_dataset/results_tabpfn_fixed/split_indices.json
  Train:  46 participants, 968 trials (ASD=22, TD=24)
  Holdout: 11 participants, 210 trials (ASD=5, TD=6)

After imputation (train medians only):
  Train missing:  0
  Holdout missing: 0

```

## ✓ Cell 9 — Model factory and pupil scaling (Bug 6 exact-match fix)

```

# Cell 9 - TabPFN factory and pupil scaler
#
# BUG 6 FIX - Pupil column exact match:
#   OLD: any(p in f for p in PUPIL_COLS) <- substring, 'pupil_mean' matches 'pupil_mean_norm'
#   NEW: f in PUPIL_COLS_EXACT          <- exact set membership

from tabpfn import TabPFNClassifier

def make_tabpfn(n_ensemble=N_ENSEMBLE):
    return TabPFNClassifier(
        n_estimators=n_ensemble,
        device='cuda',
        ignore_pretraining_limits=True,
    )

def apply_pupil_scaling(X, feat_cols_list, scaler=None, fit=False):
    """
    BUG 6 FIX: exact set membership instead of substring match.
    Returns (X_scaled, scaler, pupil_idx).
    """
    pupil_idx = [i for i, f in enumerate(feat_cols_list) if f in PUPIL_COLS_EXACT]
    if not pupil_idx:
        return X, scaler, pupil_idx
    X = X.copy()
    if fit or scaler is None:
        scaler = StandardScaler()
        X[:, pupil_idx] = scaler.fit_transform(X[:, pupil_idx])
    else:
        X[:, pupil_idx] = scaler.transform(X[:, pupil_idx])
    return X, scaler, pupil_idx

```

```
print('TabPFN factory and scaler defined OK')
print(f'Pupil columns for exact-match scaling: {sorted(PUPIL_COLS_EXACT)}')
```

```
TabPFN factory and scaler defined OK
Pupil columns for exact-match scaling: ['pupil_asymmetry_mean', 'pupil_asymmetry_std', 'pupil_l_mean', 'pupil_l_std', 'pupil_r_mean', 'pupil_r_std', 'pupil_slope']
```

## ✓ Cell 10 — Statistical feature validation (FDR, BH correction)

```
# Cell 10 — FDR feature validation
# Used ONLY inside each LOOCV fold (Bug 5 fix — no global pre-filter)

from scipy.stats import shapiro, mannwhitneyu, ttest_ind
from statsmodels.stats.multitest import multipletests

def validate_features_fdr(X_part_means, y_part, alpha=ALPHA):
    """
    Per-feature Mann-Whitney U or Welch t-test with Benjamini-Hochberg FDR.
    X_part_means: DataFrame of participant-level feature means (n_participants x n_features)
    y_part: array of labels (length n_participants)
    Returns: (sig_features, effect_sizes, p_raw, p_adj)
    """
    cols = X_part_means.columns.tolist()
    p_raw_arr = []
    effect = {}

    for col in cols:
        asd = X_part_means[col][y_part == 1].dropna().values
        td = X_part_means[col][y_part == 0].dropna().values
        if len(asd) < 3 or len(td) < 3:
            p_raw_arr.append(1.0)
            effect[col] = 0.0
            continue
        try:
            _, p_a = shapiro(asd)
            _, p_t = shapiro(td)
        except Exception:
            p_a, p_t = 0.0, 0.0
        if p_a < 0.05 or p_t < 0.05:
            stat, p = mannwhitneyu(asd, td, alternative='two-sided')
            r = float(1 - (2 * stat) / (len(asd) * len(td)))
        else:
            _, p = ttest_ind(asd, td, equal_var=False)
            pooled_std = np.sqrt((asd.std()**2 + td.std()**2) / 2.0 + 1e-12)
            r = float((asd.mean() - td.mean()) / pooled_std)
        p_raw_arr.append(float(p))
        effect[col] = r

    reject, p_adj_arr, _, _ = multipletests(p_raw_arr, alpha=alpha, method='fdr_bh')
    p_raw = dict(zip(cols, p_raw_arr))
    p_adj = dict(zip(cols, p_adj_arr))
    sig_features = [c for c, r in zip(cols, reject) if r]
```

```
return sig_features, effect, p_raw, p_adj
```

```
print('FDR feature validation defined OK')
```

```
FDR feature validation defined OK
```

## ✓ Cell 11 — LOOCV with per-fold feature selection (Bug 5 fix)

```
# Cell 11 - LOOCV
#
# BUG 5 FIX - Feature selection leakage:
# OLD: global_feature_validation(df_train, feat_cols) run ONCE before LOOCV,
#      using labels of ALL participants including the to-be-left-out participant.
# NEW: feature selection (FDR) runs INSIDE each fold on the fold's train split
#      only (45 participants), with the left-out participant fully excluded.
#      No pre-filter at all.
#
# BUG 6 FIX: pupil scaling uses exact column match (via apply_pupil_scaling)

from sklearn.model_selection import LeaveOneGroupOut
from sklearn.metrics import roc_auc_score, f1_score, recall_score, confusion_matrix, accuracy_score
from tqdm import tqdm

def aggregate_to_participant(participant_ids, y_true, y_prob):
    df_agg = pd.DataFrame({'participant_id': participant_ids,
                           'y_true': y_true, 'y_prob': y_prob})
    agg = df_agg.groupby('participant_id').agg(
        y_true      = ('y_true', 'first'),
        y_prob_mean = ('y_prob', 'mean'),
        n_trials    = ('y_prob', 'count'),
    ).reset_index()
    agg['y_pred'] = (agg['y_prob_mean'] >= 0.5).astype(int)
    agg['group']  = agg['y_true'].map({1: 'ASD', 0: 'TD'})
    agg['predicted'] = agg['y_pred'].map({1: 'ASD', 0: 'TD'})
    agg['correct']  = (agg['y_true'] == agg['y_pred']).astype(int)
    return agg

def run_loocv(df_train, feat_cols_all):
    """
    Leave-One-Participant-Out CV.
    Feature selection (FDR) is done INSIDE each fold on fold-train only.
    """
    logo = LeaveOneGroupOut()
    groups = df_train['participant_id'].values
    y = df_train['label'].values.astype(int)
    X = df_train[feat_cols_all].values

    all_ids, all_true, all_probs = [], [], []
    fold_rows = []
    n_parts = df_train['participant_id'].nunique()
```

```

print(f'Running LOOCV ({n_parts} folds) n_estimators={N_ENSEMBLE}')
print(f'{"Fold":<6}{ "ID":<14}{ "Group":<7}{ "P(ASD)":<10}{ "OK":<5}{ "SigF"}')
print('-' * 48)

folds = list(logo.split(X, y, groups))
pbar = tqdm(folds, desc=' LOOCV', unit='fold')

for fi, (tr_idx, te_idx) in enumerate(pbar):
    left_id = groups[te_idx[0]]
    left_label = y[te_idx[0]]

    df_tr = df_train.iloc[tr_idx].copy()
    df_te = df_train.iloc[te_idx].copy()
    y_te = y[te_idx].astype(int)

    # BUG 5 FIX: feature selection inside fold, on fold-train only
    pm = df_tr.groupby('participant_id')[feat_cols_all].mean()
    pl = df_tr.groupby('participant_id')['label'].first()
    sig, _, _ = validate_features_fdr(pm, pl.values)
    if len(sig) == 0:
        sig = feat_cols_all # fallback: use all

    Xt = df_tr[sig].values.astype(float)
    y_tr = df_tr['label'].values.astype(int)
    Xv = df_te[sig].values.astype(float)

    Xt, sc, _ = apply_pupil_scaling(Xt, sig, fit=True)
    Xv, _, _ = apply_pupil_scaling(Xv, sig, scaler=sc)

    model = make_tabPFN()
    model.fit(Xt, y_tr)
    probs = model.predict_proba(Xv)[: , 1]

    all_ids.extend(groups[te_idx].tolist())
    all_true.extend(y_te.tolist())
    all_probs.extend(probs.tolist())

    prob_mean = float(np.mean(probs))
    pred = int(prob_mean >= 0.5)
    ok = 'Y' if pred == left_label else 'N'
    group = 'ASD' if left_label == 1 else 'TD'

    fold_rows.append({
        'fold': fi+1, 'participant_id': left_id,
        'true_label': left_label, 'group': group,
        'n_train_trials': len(tr_idx), 'n_test_trials': len(y_te),
        'n_sig_feats': len(sig),
        'prob_asd': round(prob_mean, 4), 'pred': pred,
        'correct': int(pred == left_label),
    })
    print(f'{"fi+1:<6}{str(left_id):<14}{group:<7}{prob_mean:<10.3f}{ok:<5}{len(sig)}')

loocv_df = pd.DataFrame(fold_rows)
part_loocv = aggregate_to_participant(
    np.array(all_ids), np.array(all_true), np.array(all_probs))
return loocv_df, part_loocv

```

```
print('LOOCV function defined OK')
```

```
LOOCV function defined OK
```

## ✓ Cell 12 — Bootstrap CI helper

```
# Cell 12 – Bootstrap confidence intervals

def bootstrap_metrics(y_true, y_prob, n_boot=N_BOOT, seed=RANDOM_STATE):
    """
    Bootstrap 95% CIs for AUC, sensitivity, specificity, F1, accuracy.
    Resamples participants (not trials) to respect independence.
    """
    rng = np.random.default_rng(seed)
    y_true = np.array(y_true)
    y_prob = np.array(y_prob)
    y_pred = (y_prob >= 0.5).astype(int)
    n = len(y_true)

    boot = {'auc': [], 'sens': [], 'spec': [], 'f1': [], 'acc': []}
    for _ in range(n_boot):
        idx = rng.choice(n, n, replace=True)
        yt, yp, ypr = y_true[idx], y_pred[idx], y_prob[idx]
        if len(np.unique(yt)) < 2:
            continue
        boot['auc'].append(roc_auc_score(yt, ypr))
        boot['sens'].append(recall_score(yt, yp, pos_label=1, zero_division=0))
        boot['spec'].append(recall_score(yt, yp, pos_label=0, zero_division=0))
        boot['f1'].append(f1_score(yt, yp, zero_division=0))
        boot['acc'].append(accuracy_score(yt, yp))

    point = {
        'auc': roc_auc_score(y_true, y_prob),
        'sens': recall_score(y_true, y_pred, pos_label=1, zero_division=0),
        'spec': recall_score(y_true, y_pred, pos_label=0, zero_division=0),
        'f1': f1_score(y_true, y_pred, zero_division=0),
        'acc': accuracy_score(y_true, y_pred),
    }
    result = {}
    for k, vals in boot.items():
        lo, hi = np.percentile(vals, [2.5, 97.5]) if vals else (0, 0)
        result[k] = (round(point[k], 4), round(lo, 4), round(hi, 4))
    return result

def print_metrics_with_ci(ci_dict, title, n):
    print(f'\n{"-"*55}')
    print(f' {title} (n={n} participants)')
    print(f'\n{"-"*55}')
    names = {'auc': 'AUC', 'sens': 'Sensitivity', 'spec': 'Specificity', 'f1': 'F1', 'acc': 'Accuracy'}
    for k, (pt, lo, hi) in ci_dict.items():
        print(f' {names.get(k,k):<16} {pt:>8.3f} [{lo:.3f}, {hi:.3f}]')
```

```
print('Bootstrap CI helper defined OK')
```

```
Bootstrap CI helper defined OK
```

## Cell 13 — Run LOOCV

```
# Cell 13 — Run LOOCV (primary result)
# Expected runtime: ~4-10 minutes depending on dataset size and n_estimators

import time
from sklearn.metrics import classification_report

feat_cols_all = [c for c in df_train.columns if c not in META_COLS]

t1 = time.time()
loocv_df, part_loocv = run_loocv(df_train, feat_cols_all)
elapsed_loocv = time.time() - t1
print(f'\nLOOCV wall time: {elapsed_loocv/60:.1f} min')

# Bootstrap CIs
loocv_ci = bootstrap_metrics(part_loocv['y_true'], part_loocv['y_prob_mean'])
print_metrics_with_ci(loocv_ci, 'LOOCV results [PRIMARY RESULT]', len(part_loocv))

# Per-class accuracy
asd_r = part_loocv[part_loocv['y_true']==1]
td_r = part_loocv[part_loocv['y_true']==0]
print(f'\n ASD: {int(asd_r["correct"].sum())/len(asd_r)} correct (mean P(ASD) = {asd_r["y_prob_mean"].mean():.3f})')
print(f' TD: {int(td_r["correct"].sum())/len(td_r)} correct (mean P(ASD) = {td_r["y_prob_mean"].mean():.3f})')

# Confusion matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(part_loocv['y_true'], part_loocv['y_pred'])
print(f'\n Confusion matrix (LOOCV):')
print(f'          Pred TD   Pred ASD')
print(f' Actual TD      {cm[0,0]:<10} {cm[0,1]:<10}')
print(f' Actual ASD      {cm[1,0]:<10} {cm[1,1]:<10}')

cr = classification_report(part_loocv['y_true'], part_loocv['y_pred'],
                           target_names=['TD', 'ASD'], digits=3)
print(f'\n Classification report (LOOCV):')
for line in cr.splitlines():
    print(f'    {line}')

# Save
loocv_df.to_csv(os.path.join(OUTPUT_DIR, 'loocv_fold_results.csv'), index=False)
part_loocv.to_csv(os.path.join(OUTPUT_DIR, 'loocv_participant_predictions.csv'), index=False)
pd.DataFrame([{'metric':k, 'value':pt, 'ci_lower':lo, 'ci_upper':hi}
               for k,(pt,lo,hi) in loocv_ci.items()]).to_csv(
    os.path.join(OUTPUT_DIR, 'loocv_summary_with_ci.csv'), index=False)
print('\nLOOCV results saved.')
```

|             |             |                                 |    |    |     |       |   |    |
|-------------|-------------|---------------------------------|----|----|-----|-------|---|----|
| L00CV: 46%  | <div></div> | 21/46 [01:00<01:13, 2.96s/fold] | 21 | 33 | TD  | 0.146 | Y | 15 |
| L00CV: 48%  | <div></div> | 22/46 [01:03<01:12, 3.00s/fold] | 22 | 34 | TD  | 0.163 | Y | 15 |
| L00CV: 50%  | <div></div> | 23/46 [01:06<01:10, 3.05s/fold] | 23 | 36 | TD  | 0.163 | Y | 15 |
| L00CV: 52%  | <div></div> | 24/46 [01:09<01:02, 2.85s/fold] | 24 | 37 | TD  | 0.099 | Y | 14 |
| L00CV: 54%  | <div></div> | 25/46 [01:11<00:58, 2.78s/fold] | 25 | 39 | TD  | 0.251 | Y | 15 |
| L00CV: 57%  | <div></div> | 26/46 [01:14<00:54, 2.73s/fold] | 26 | 42 | TD  | 0.675 | N | 15 |
| L00CV: 59%  | <div></div> | 27/46 [01:17<00:51, 2.70s/fold] | 27 | 43 | TD  | 0.188 | Y | 14 |
| L00CV: 61%  | <div></div> | 28/46 [01:20<00:51, 2.83s/fold] | 28 | 44 | TD  | 0.028 | Y | 15 |
| L00CV: 63%  | <div></div> | 29/46 [01:23<00:47, 2.79s/fold] | 29 | 45 | TD  | 0.154 | Y | 15 |
| L00CV: 65%  | <div></div> | 30/46 [01:25<00:43, 2.70s/fold] | 30 | 46 | TD  | 0.226 | Y | 14 |
| L00CV: 67%  | <div></div> | 31/46 [01:28<00:40, 2.70s/fold] | 31 | 47 | TD  | 0.150 | Y | 15 |
| L00CV: 70%  | <div></div> | 32/46 [01:30<00:37, 2.66s/fold] | 32 | 48 | TD  | 0.417 | Y | 14 |
| L00CV: 72%  | <div></div> | 33/46 [01:33<00:35, 2.76s/fold] | 33 | 49 | TD  | 0.237 | Y | 14 |
| L00CV: 74%  | <div></div> | 34/46 [01:36<00:32, 2.75s/fold] | 34 | 50 | TD  | 0.109 | Y | 15 |
| L00CV: 76%  | <div></div> | 35/46 [01:38<00:29, 2.65s/fold] | 35 | 51 | TD  | 0.072 | Y | 14 |
| L00CV: 78%  | <div></div> | 36/46 [01:41<00:26, 2.65s/fold] | 36 | 52 | TD  | 0.248 | Y | 14 |
| L00CV: 80%  | <div></div> | 37/46 [01:44<00:23, 2.59s/fold] | 37 | 54 | TD  | 0.130 | Y | 14 |
| L00CV: 83%  | <div></div> | 38/46 [01:47<00:21, 2.71s/fold] | 38 | 55 | TD  | 0.421 | Y | 14 |
| L00CV: 85%  | <div></div> | 39/46 [01:50<00:19, 2.83s/fold] | 39 | 56 | TD  | 0.222 | Y | 15 |
| L00CV: 87%  | <div></div> | 40/46 [01:52<00:16, 2.71s/fold] | 40 | 57 | TD  | 0.172 | Y | 14 |
| L00CV: 89%  | <div></div> | 41/46 [01:55<00:13, 2.64s/fold] | 41 | 58 | TD  | 0.241 | Y | 14 |
| L00CV: 91%  | <div></div> | 42/46 [01:57<00:10, 2.65s/fold] | 42 | 59 | TD  | 0.320 | Y | 15 |
| L00CV: 93%  | <div></div> | 43/46 [02:00<00:08, 2.74s/fold] | 43 | 6  | ASD | 0.680 | Y | 15 |
| L00CV: 96%  | <div></div> | 44/46 [02:03<00:05, 2.87s/fold] | 44 | 7  | ASD | 0.924 | Y | 15 |
| L00CV: 98%  | <div></div> | 45/46 [02:06<00:02, 2.81s/fold] | 45 | 8  | ASD | 0.886 | Y | 15 |
| L00CV: 100% | <div></div> | 46/46 [02:09<00:00, 2.81s/fold] | 46 | 9  | ASD | 0.912 | Y | 15 |

L00CV wall time: 2.2 min

L00CV results [PRIMARY RESULT] (n=46 participants)

|             |       |                |
|-------------|-------|----------------|
| AUC         | 0.966 | [0.911, 1.000] |
| Sensitivity | 0.818 | [0.647, 0.958] |
| Specificity | 0.958 | [0.862, 1.000] |
| F1          | 0.878 | [0.757, 0.974] |
| Accuracy    | 0.891 | [0.804, 0.978] |

ASD: 18/22 correct (mean P(ASD) = 0.679)  
TD: 23/24 correct (mean P(ASD) = 0.228)

Confusion matrix (L00CV):

|            |         |          |
|------------|---------|----------|
|            | Pred TD | Pred ASD |
| Actual TD  | 23      | 1        |
| Actual ASD | 4       | 18       |

Classification report (L00CV):

|              |           |        |          |         |
|--------------|-----------|--------|----------|---------|
|              | precision | recall | f1-score | support |
| TD           | 0.852     | 0.958  | 0.902    | 24      |
| ASD          | 0.947     | 0.818  | 0.878    | 22      |
| accuracy     |           |        | 0.891    | 46      |
| macro avg    | 0.900     | 0.888  | 0.890    | 46      |
| weighted avg | 0.898     | 0.891  | 0.891    | 46      |

L00CV results saved.

## Cell 14 — Permutation test

```
# Cell 14 – Permutation test (null AUC distribution)
# N_PERM=100 for dev (~5 min). Change to 1000 for camera-ready paper.

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def permutation_test(df_train, feat_cols_all, observed_auc, n_perm=N_PERM):
    y_part = df_train.groupby('participant_id')['label'].first()
    parts = list(y_part.index)
    y_arr = y_part.values.astype(int)
    rng = np.random.default_rng(RANDOM_STATE)

    print(f'Permutation test: {n_perm} shuffles ...')
    if n_perm == 0:
        print('Skipped (N_PERM=0).')
        return np.array([]), float('nan')

    null_aucs = []
    pbar = tqdm(range(n_perm), desc=' Permutations', unit='perm')
    for _ in pbar:
        y_shuffled = rng.permutation(y_arr)
        label_map = dict(zip(parts, y_shuffled))
        df_perm = df_train.copy()
        df_perm['label'] = df_perm['participant_id'].map(label_map)

        n = len(parts)
        tr = rng.choice(n, int(n * 0.8), replace=False)
        te = np.setdiff1d(np.arange(n), tr)
        if len(np.unique(y_shuffled[te])) < 2:
            continue
        tr_parts = [parts[j] for j in tr]
        te_parts = [parts[j] for j in te]
        df_tr = df_perm[df_perm['participant_id'].isin(tr_parts)]
        df_te = df_perm[df_perm['participant_id'].isin(te_parts)]

        Xt = df_tr[feat_cols_all].values.astype(float)
        Xv = df_te[feat_cols_all].values.astype(float)
        yt = df_tr['label'].values.astype(int)
        yv_true = np.array([y_shuffled[parts.index(p)]
                             for p in df_te['participant_id'].values])

        Xt, sc, _ = apply_pupil_scaling(Xt, feat_cols_all, fit=True)
        Xv, _, _ = apply_pupil_scaling(Xv, feat_cols_all, scaler=sc)
        m = make_tabPFN(n_ensemble=8)
        m.fit(Xt, yt)
        probs = m.predict_proba(Xv)[: , 1]
        part_probs = (pd.DataFrame({'pid': df_te['participant_id'].values,
                                    'p': probs, 'yt': yv_true})
                      .groupby('pid').agg(p=('p', 'mean'), yt=('yt', 'first')))
        if len(np.unique(part_probs['yt'])) < 2:
```

```

        continue
    null_aucs.append(roc_auc_score(part_probs['yt'], part_probs['p']))
    if len(null_aucs) > 0:
        pbar.set_postfix({'null_mean': f'{np.mean(null_aucs):.3f}',
                          'obs': f'{observed_auc:.3f}'})

null_aucs = np.array(null_aucs)
emp_p = float((null_aucs >= observed_auc).sum() / len(null_aucs)) \
    if len(null_aucs) > 0 else float('nan')
print(f'\n null AUC: mean={null_aucs.mean():.3f}, std={null_aucs.std():.3f}')
print(f' Observed AUC={observed_auc:.3f} -> p={emp_p:.4f}')
return null_aucs, emp_p

observed_auc = loocv_ci['auc'][0]
null_aucs, emp_p = permutation_test(df_train, feat_cols_all, observed_auc)

# Plot
if len(null_aucs) > 0:
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.hist(null_aucs, bins=30, color='#90CAF9', edgecolor='white')
    ax.axvline(observed_auc, color='#D32F2F', linewidth=2,
               label=f'Observed AUC={observed_auc:.3f} (p={emp_p:.4f})')
    ax.set_xlabel('AUC under label permutation'); ax.set_ylabel('Count')
    ax.set_title('Permutation test – null distribution'); ax.legend()
    plt.tight_layout()
    perm_plot_path = os.path.join(OUTPUT_DIR, 'permutation_null_distribution.png')
    plt.savefig(perm_plot_path, dpi=300, bbox_inches='tight')
    plt.show()
    print(f'Saved: {perm_plot_path}')

# Save
pd.DataFrame({
    'observed_auc': [observed_auc],
    'n_permutations': [len(null_aucs)],
    'null_auc_mean': [round(null_aucs.mean(), 4) if len(null_aucs) else np.nan],
    'null_auc_std': [round(null_aucs.std(), 4) if len(null_aucs) else np.nan],
    'empirical_p': [round(emp_p, 4)],
    'significant': [emp_p < ALPHA if not np.isnan(emp_p) else 'skipped'],
}).to_csv(os.path.join(OUTPUT_DIR, 'permutation_test_results.csv'), index=False)

Permutation test: 100 shuffles ...
Permutations: 100%|██████████| 100/100 [01:55<00:00, 1.16s/perm, null_mean=0.490, obs=0.966]

null AUC: mean=0.490, std=0.206
Observed AUC=0.966 -> p=0.0200
Saved: /content/drive/MyDrive/asd_eye_dataset/results_tabpfn_fixed/permutation_null_distribution.png

```

## ✧ Cell 15 — Train final model and save bundle

```

# Cell 15 – Train final model on ALL train participants
# Feature selection is run one final time on all train participants
# (this is for the final model only, not for any evaluation metric)

import joblib

```

```

# Final FDR feature selection on full train set (for final model only)
pm_all = df_train.groupby('participant_id')[feat_cols_all].mean()
pl_all = df_train.groupby('participant_id')['label'].first()
significant, effect_sizes, p_raw, p_adj = validate_features_fdr(pm_all, pl_all.values)
if len(significant) == 0:
    print('WARNING: no features survived FDR - using all features')
    significant = feat_cols_all

print(f'Features surviving FDR: {len(significant)} / {len(feat_cols_all)}')
print(f' {significant}')

# Save FDR table
val_df = pd.DataFrame([
    'feature': f,
    'p_raw': round(p_raw[f], 5),
    'p_adj_fdr': round(p_adj[f], 5),
    'effect_r': round(effect_sizes.get(f, 0.0), 4),
    'significant': f in significant
] for f in feat_cols_all]).sort_values('p_adj_fdr')
val_df.to_csv(os.path.join(OUTPUT_DIR, 'statistical_validation_fdr.csv'), index=False)
print('\nTop 10 features by FDR-adjusted p:')
print(val_df.head(10)[['feature', 'p_raw', 'p_adj_fdr', 'effect_r', 'significant']].to_string(index=False))

# Train final model
X_final = df_train[significant].values.astype(float)
y_final = df_train['label'].values.astype(int)
X_final, final_scaler, pupil_idx = apply_pupil_scaling(X_final, significant, fit=True)

final_model = make_tabPFN(n_ensemble=N_ENSEMBLE)
final_model.fit(X_final, y_final)
print(f'\nFinal model trained on {df_train["participant_id"].nunique()} participants, '
      f'{len(df_train)} trials, {len(significant)} features')

# Save bundle
bundle = {
    'model': final_model,
    'scaler': final_scaler,
    'feature_names': significant,
    'pupil_idx': pupil_idx,
    'train_means': dict(zip(significant, df_train[significant].mean().values)),
    'train_std': dict(zip(significant, df_train[significant].std().values)),
    'train_medians': dict(zip(feat_cols_all, train_medians.values)),
    'model_type': 'TabPFN-v2.5',
    'n_estimators': N_ENSEMBLE,
    'n_train_parts': df_train['participant_id'].nunique(),
    'n_train_trials': len(df_train),
}
bundle_path = os.path.join(OUTPUT_DIR, 'asd_model_tabPFN_final.joblib')
joblib.dump(bundle, bundle_path)
print(f'Bundle saved: {bundle_path}')

Features surviving FDR: 15 / 28
['fix_count', 'fix_dur_mean_ms', 'fix_dur_std_ms', 'fix_dur_max_ms', 'fix_dur_cv', 'fix_rate_per_min', 'fix_dispersion_mean', 'fix_seq_norm', 'sac_count', 'sac_amplitude_mean', 'sac_rate']

Top 10 features by FDR-adjusted p:
feature p_raw p_adj_fdr effect_r significant

```

|                    |         |         |         |      |
|--------------------|---------|---------|---------|------|
| sac_rate_per_min   | 0.00001 | 0.00014 | 1.6655  | True |
| fix_rate_per_min   | 0.00002 | 0.00029 | -0.7348 | True |
| fix_seq_norm       | 0.00004 | 0.00035 | -0.7121 | True |
| fix_count          | 0.00009 | 0.00045 | 0.6742  | True |
| fix_dur_cv         | 0.00010 | 0.00045 | -1.3022 | True |
| fix_dur_max_ms     | 0.00007 | 0.00045 | 0.6856  | True |
| blink_rate_per_min | 0.00011 | 0.00045 | -0.6667 | True |
| sac_count          | 0.00030 | 0.00104 | 0.6250  | True |
| fix_dur_std_ms     | 0.00074 | 0.00229 | 0.5833  | True |
| blink_long_ratio   | 0.00111 | 0.00312 | 1.0704  | True |

Final model trained on 46 participants, 968 trials, 15 features

Bundle saved: /content/drive/MyDrive/asd\_eye\_dataset/results\_tabpfn\_fixed/asd\_model\_tabpfn\_final.joblib

## ✓ Cell 16 — Feature importance (permutation + SHAP)

# Cell 16 – Feature importance

N\_REPEATS\_IMP = 5 # raise to 10 for paper

```
X_imp = df_train[significant].values.astype(float).copy()
X_imp, _, _ = apply_pupil_scaling(X_imp, significant, scaler=final_scaler)
y_imp = df_train['label'].values.astype(int)
```

```
baseline = roc_auc_score(y_imp, final_model.predict_proba(X_imp)[: , 1])
n_feats = len(significant)
imp_means = np.zeros(n_feats)
imp_stds = np.zeros(n_feats)
```

```
print(f'Permutation importance ({N_REPEATS_IMP} repeats × {n_feats} features) ...')
```

```
pbar_imp = tqdm(range(n_feats), desc=' Importance', unit='feat')
```

```
for fi in pbar_imp:
```

```
    scores = []
```

```
    X_perm = X_imp.copy()
```

```
    for rep in range(N_REPEATS_IMP):
```

```
        rng_i = np.random.default_rng(RANDOM_STATE + fi * 100 + rep)
```

```
        X_perm[:, fi] = rng_i.permutation(X_imp[:, fi])
```

```
        prob = final_model.predict_proba(X_perm)[: , 1]
```

```
        scores.append(baseline - roc_auc_score(y_imp, prob))
```

```
        X_perm[:, fi] = X_imp[:, fi]
```

```
    imp_means[fi] = np.mean(scores)
```

```
    imp_stds[fi] = np.std(scores)
```

```
    pbar_imp.set_postfix({'feat': significant[fi][:18], 'drop': f'{imp_means[fi]:.3f}'})
```

```
perm_df = pd.DataFrame({
```

```
    'feature': significant,
```

```
    'perm_imp_mean': imp_means.round(4),
```

```
    'perm_imp_std': imp_stds.round(4),
```

```
}).sort_values('perm_imp_mean', ascending=False)
```

```
perm_df.to_csv(os.path.join(OUTPUT_DIR, 'feature_importance.csv'), index=False)
```

```
# Plot
```

```
top = perm_df.head(min(15, len(perm_df)))
```

```
fig, ax = plt.subplots(figsize=(10, 6))
```

```

ax.barh(top['feature'][:-1], top['perm_imp_mean'][:-1],
        xerr=top['perm_imp_std'][:-1], color='#2196F3', capsize=3)
ax.set_xlabel('Permutation importance (AUC drop)')
ax.set_title('Feature importance - permutation')
plt.tight_layout()
imp_plot = os.path.join(OUTPUT_DIR, 'importance_permutation.png')
plt.savefig(imp_plot, dpi=300, bbox_inches='tight')
plt.show()
print(f'Importance saved: {imp_plot}')

# SHAP beeswarm
try:
    import shap
    print('\nSHAP kernel explainer (background=50, nsamples=100) ...')
    background = shap.kmeans(X_imp, min(50, len(X_imp)))
    explainer = shap.KernelExplainer(lambda x: final_model.predict_proba(x)[: , 1], background)
    rng_s = np.random.default_rng(RANDOM_STATE)
    idx = rng_s.choice(len(X_imp), min(200, len(X_imp)), replace=False)
    shap_vals = explainer.shap_values(X_imp[idx], nsamples=100, l1_reg='aic')

    plt.figure(figsize=(10, 7))
    shap.summary_plot(shap_vals, X_imp[idx], feature_names=significant,
                     plot_type='dot', show=False)
    plt.title('SHAP beeswarm - feature direction of effect')
    plt.tight_layout()
    shap_path = os.path.join(OUTPUT_DIR, 'shap_be swarm.png')
    plt.savefig(shap_path, dpi=300, bbox_inches='tight')
    plt.show()
    print(f'SHAP beeswarm saved: {shap_path}')
except ImportError:
    print('SHAP not installed. pip install shap to enable.')
except Exception as e:
    print(f'SHAP skipped: {e}')

Permutation importance (5 repeats x 15 features) ...
Importance: 100%|██████████| 15/15 [04:36<00:00, 18.45s/feat, feat=gaze_x_mean, drop=0.002]
Importance saved: /content/drive/MyDrive/asd_eye_dataset/results_tabpfn_fixed/importance_permutation.png

SHAP kernel explainer (background=50, nsamples=100) ...
100% 200/200 [39:48<00:00, 11.98s/it]
SHAP beeswarm saved: /content/drive/MyDrive/asd_eye_dataset/results_tabpfn_fixed/shap_be swarm.png

```

## ✓ Cell 17 — Holdout evaluation (one-time, locked)

```

# Cell 17 - Holdout evaluation
# This cell writes a .holdout_used flag after first run.
# Subsequent runs are blocked to prevent p-hacking.
# Delete .holdout_used ONLY if you have a genuinely new split.

from sklearn.metrics import roc_curve

def plot_confusion_matrix_fig(y_true, y_pred, path):

```

```

from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true, y_pred)
fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(cm, cmap='Blues')
plt.colorbar(im, ax=ax)
for i in range(2):
    for j in range(2):
        ax.text(j, i, str(cm[i, j]), ha='center', va='center', fontsize=16,
                fontweight='bold', color='white' if cm[i, j] > cm.max()/2 else 'black')
ax.set_xticks([0,1]); ax.set_yticks([0,1])
ax.set_xticklabels(['TD', 'ASD']); ax.set_yticklabels(['TD', 'ASD'])
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
ax.set_title(f'Confusion matrix - holdout (n={len(y_true)})')
plt.tight_layout()
plt.savefig(path, dpi=300, bbox_inches='tight'); plt.show()

def plot_roc_fig(y_true, y_prob, auc, path):
    fpr, tpr, _ = roc_curve(y_true, y_prob)
    fig, ax = plt.subplots(figsize=(5, 5))
    ax.plot(fpr, tpr, color='#2196F3', linewidth=2, label=f'TabPFN AUC={auc:.3f}')
    ax.plot([0,1],[0,1], 'k--', linewidth=1, alpha=0.5, label='Random')
    ax.set_xlabel('False positive rate'); ax.set_ylabel('True positive rate')
    ax.set_title(f'ROC curve - holdout (n={len(y_true)})')
    ax.legend(); ax.set_xlim([0,1]); ax.set_ylim([0,1])
    plt.tight_layout()
    plt.savefig(path, dpi=300, bbox_inches='tight'); plt.show()

if os.path.exists(HOLDOUT_FLAG):
    print('*** HOLDOUT ALREADY USED - skipping to prevent p-hacking ***')
    print('(Delete .holdout_used only if you have a new split.)')
else:
    # Apply SAME feature set as final model
    # Fill missing features with training means (from bundle)
    for f in significant:
        if f not in df_holdout.columns:
            df_holdout[f] = bundle['train_means'].get(f, 0.0)

    X_hold = df_holdout[significant].values.astype(float)
    y_hold = df_holdout['label'].values.astype(int)
    X_hold, _, _ = apply_pupil_scaling(X_hold, significant, scaler=final_scaler)

    probs_hold = final_model.predict_proba(X_hold)[ :, 1]
    part_holdout = aggregate_to_participant(
        df_holdout['participant_id'].values, y_hold, probs_hold)

    # Per-participant table
    print('\nPer-participant holdout results:')
    print(f'{"ID":<14>{"Group":<8>{"P(ASD)":<10>{"Predicted":<12>{"Correct":<12}'
    print('-' * 52)
    for _, row in part_holdout.sort_values('y_true', ascending=False).iterrows():
        tick = '✓' if row['correct'] else 'X'
        print(f'{str(row["participant_id"]):<14}{row["group"]:<8}'
              f'{row["y_prob_mean"]:<10.3f}{row["predicted"]:<12}{tick}')

```

```

hold_ci = bootstrap_metrics(part_holdout['y_true'], part_holdout['y_prob_mean'])
print_metrics_with_ci(hold_ci, 'Holdout results', len(part_holdout))

cm_h = confusion_matrix(part_holdout['y_true'], part_holdout['y_pred'])
print(f'\n Confusion matrix (holdout):')
print(f'                Pred TD   Pred ASD')
print(f' Actual TD          {cm_h[0,0]:<10}  {cm_h[0,1]:<10}')
print(f' Actual ASD          {cm_h[1,0]:<10}  {cm_h[1,1]:<10}')

# Plots
plot_confusion_matrix_fig(part_holdout['y_true'], part_holdout['y_pred'],
                           os.path.join(OUTPUT_DIR, 'holdout_confusion_matrix.png'))
plot_roc_fig(part_holdout['y_true'], part_holdout['y_prob_mean'],
              hold_ci['auc'][0], os.path.join(OUTPUT_DIR, 'holdout_roc_curve.png'))

# Save outputs
part_holdout.to_csv(os.path.join(OUTPUT_DIR, 'holdout_predictions.csv'), index=False)
pd.DataFrame([{'metric':k, 'value':pt, 'ci_lower':lo, 'ci_upper':hi}
               for k,(pt,lo,hi) in hold_ci.items()]).to_csv(
    os.path.join(OUTPUT_DIR, 'holdout_summary_with_ci.csv'), index=False)

# Write flag
with open(HOLDOUT_FLAG, 'w') as f:
    f.write(datetime.datetime.now().isoformat())

# Final summary
auc_pt, auc_lo, auc_hi = hold_ci['auc']
print(f'\n{"="*55}')
print(f'  FINAL HOLDOUT RESULT')
print(f'  AUC  {auc_pt:.3f}  [{auc_lo:.3f}, {auc_hi:.3f}]')
print(f'  Sensitivity  {hold_ci["sens"][0]:.3f}')
print(f'  Specificity  {hold_ci["spec"][0]:.3f}')
print(f'  Permutation p = {emp_p:.4f}')
if auc_pt >= 0.90:
    print('  ✓ Strong holdout result.')
elif auc_pt >= 0.80:
    print('  ~ Good holdout result.')
else:
    print('  ! Investigate – check for bugs above.')
print(f'{"="*55}')
```

Per-participant holdout results:

| ID | Group | P(ASD) | Predicted | Correct |
|----|-------|--------|-----------|---------|
| 18 | ASD   | 0.761  | ASD       | ✓       |
| 27 | ASD   | 0.797  | ASD       | ✓       |
| 29 | ASD   | 0.747  | ASD       | ✓       |
| 4  | ASD   | 0.915  | ASD       | ✓       |
| 5  | ASD   | 0.846  | ASD       | ✓       |
| 35 | TD    | 0.280  | TD        | ✓       |
| 31 | TD    | 0.183  | TD        | ✓       |
| 38 | TD    | 0.482  | TD        | ✓       |
| 40 | TD    | 0.146  | TD        | ✓       |
| 41 | TD    | 0.354  | TD        | ✓       |
| 53 | TD    | 0.183  | TD        | ✓       |

Holdout results (n=11 participants)

|             |       |                |
|-------------|-------|----------------|
| AUC         | 1.000 | [1.000, 1.000] |
| Sensitivity | 1.000 | [1.000, 1.000] |
| Specificity | 1.000 | [1.000, 1.000] |
| F1          | 1.000 | [1.000, 1.000] |
| Accuracy    | 1.000 | [1.000, 1.000] |

Confusion matrix (holdout):

|            | Pred TD | Pred ASD |
|------------|---------|----------|
| Actual TD  | 6       | 0        |
| Actual ASD | 0       | 5        |

```
=====
FINAL HOLDOUT RESULT
AUC 1.000 [1.000, 1.000]
Sensitivity 1.000
Specificity 1.000
Permutation p = 0.0200
✓ Strong holdout result.
=====
```

## ✓ Cell 18 — Summary of all output files

# Cell 18 – List all output files generated

```
print(f'All outputs in: {OUTPUT_DIR}\n')
expected = [
    'trial_features_tabPFN.csv',
    'qc_trial_drops.csv',
    'split_indices.json',
    'statistical_validation_fdr.csv',
    'loocv_fold_results.csv',
    'loocv_participant_predictions.csv',
    'loocv_summary_with_ci.csv',
    'permutation_test_results.csv',
    'permutation_null_distribution.png',
    'feature_importance.csv',
    'importance_permutation.png',
    'shap_beeswarm.png',
    'holdout_predictions.csv',
    'holdout_summary_with_ci.csv',
    'holdout_confusion_matrix.png',
    'holdout_roc_curve.png',
    'asd_model_tabPFN_final.joblib',
]
for fname in expected:
    p = os.path.join(OUTPUT_DIR, fname)
    tag = '✓' if os.path.exists(p) else 'X NOT GENERATED'
    size = f' ({os.path.getsize(p)/1024:.1f} KB)' if os.path.exists(p) else ''
    print(f' {tag} {fname}{size}')
```

All outputs in: /content/drive/MyDrive/asd\_eye\_dataset/results\_tabPFN\_fixed

```
✓ trial_features_tabPFN.csv (530.4 KB)
✓ qc_trial_drops.csv (4.7 KB)
```

- ✓ split\_indices.json (0.7 KB)
- ✓ statistical\_validation\_fdr.csv (1.2 KB)
- ✓ loocv\_fold\_results.csv (1.5 KB)
- ✓ loocv\_participant\_predictions.csv (1.8 KB)
- ✓ loocv\_summary\_with\_ci.csv (0.1 KB)
- ✓ permutation\_test\_results.csv (0.1 KB)
- ✓ permutation\_null\_distribution.png (72.2 KB)
- ✓ feature\_importance.csv (0.5 KB)
- ✓ importance\_permutation.png (160.8 KB)
- ✓ shap\_beeswarm.png (412.8 KB)
- ✓ holdout\_predictions.csv (0.4 KB)
- ✓ holdout\_summary\_with\_ci.csv (0.1 KB)
- ✓ holdout\_confusion\_matrix.png (51.8 KB)
- ✓ holdout\_roc\_curve.png (94.9 KB)
- ✓ asd\_model\_tabpfn\_final.joblib (47851.0 KB)

## ✓ Cell 19 — Download key files to your computer

All files are already saved to your Google Drive at `asd_eye_dataset/results_tabpfn_fixed/`.

Run this cell if you also want to download them directly to your local machine.

```
# Cell 19 - Download files directly to your computer
# Each file triggers a browser download prompt.
# Skip any file you don't need by commenting it out.

from google.colab import files
import os

# Files to download - in order of importance
to_download = [
    ('asd_model_tabpfn_final.joblib',      '← trained model bundle (load this for inference)'),
    ('split_indices.json',                 '← participant train/holdout split (keep this!)'),
    ('loocv_summary_with_ci.csv',           '← LOOCV metrics with 95% CIs'),
    ('holdout_summary_with_ci.csv',         '← holdout metrics with 95% CIs'),
    ('loocv_participant_predictions.csv',   '← per-participant LOOCV predictions'),
    ('holdout_predictions.csv',             '← per-participant holdout predictions'),
    ('statistical_validation_fdr.csv',      '← FDR feature validation table'),
    ('feature_importance.csv',              '← permutation importance scores'),
    ('permutation_test_results.csv',        '← permutation test p-value'),
    ('loocv_fold_results.csv',              '← fold-by-fold LOOCV details'),
    ('qc_trial_drops.csv',                  '← QC: dropped trials per participant'),
    ('permutation_null_distribution.png',    '← null AUC distribution plot'),
    ('importance_permutation.png',          '← feature importance bar chart'),
    ('shap_beeswarm.png',                   '← SHAP direction-of-effect plot'),
    ('holdout_confusion_matrix.png',        '← holdout confusion matrix'),
    ('holdout_roc_curve.png',               '← holdout ROC curve'),
    ('trial_features_tabpfn.csv',            '← extracted features (all participants)'),
]

print('Starting downloads...\n')
print('NOTE: All files are also already saved in your Drive at:')
print(f' {OUTPUT_DIR}\n')
print('Each download below will open a browser prompt.\n')

downloaded, skipped = [], []
```

```
for fname, desc in to_download:
    fpath = os.path.join(OUTPUT_DIR, fname)
    if os.path.exists(fpath):
        size_kb = os.path.getsize(fpath) / 1024
        print(f' Downloading: {fname} ({size_kb:.1f} KB) {desc}')
        try:
            files.download(fpath)
            downloaded.append(fname)
        except Exception as e:
            print(f' WARNING: download failed: {e}')
            skipped.append(fname)
    else:
        print(f' SKIP (not generated): {fname} {desc}')
        skipped.append(fname)

print(f'\nDone. {len(downloaded)} files downloaded, {len(skipped)} skipped.')
print('\nTo load the model later for inference:')
print(" import joblib")
print(" bundle = joblib.load('asd_model_tabpfn_final.joblib')")
print(" model      = bundle['model']")
print(" feat_cols   = bundle['feature_names']")
print(" scaler      = bundle['scaler']")
print(" probs = model.predict_proba(X_new)[: , 1] # P(ASD)")
```

Starting downloads...

Start coding or [generate](#) with AI.

/content/drive/MyDrive/asd\_eye\_dataset/results\_tabpfn\_fixed

Each download below will open a browser prompt.